The Journal of Open Source Software

openpv/simshady: A Javascript Package for Photovoltaic Yield Estimation Based on 3D Meshes

Florian Kotthoff [1,2¶], **Mino Estrella** [3], **Martin Großhauser** [1], **Konrad Heidler** [1], **and Korbinian Pöppel** [1]

1 OpenPV GbR, Munich, Germany **2** OFFIS Institute for Information Technology, Escherweg 2, 26121 Oldenburg, Germany **3** Chair of Renewable and Sustainable Energy Systems, Technical University of Munich, Germany ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

Summary

openpv/simshady (short: `simshady`) is a JavaScript package for simulating photovoltaic (PV) energy yields. It integrates local climate data and 3D objects into its shading simulation, utilizing Three.js meshes for geometric modeling. The package performs shading analysis using a WebGL-parallelized implementation of the Möller-Trumbore intersection algorithm (Möller & Trumbore, 1997). With the software, users can obtain both aggregated and spatial or temporally resolved simualted scenes of expected PV yields for further processing. `simshady` provides both a package for web development, as well as a Command-line interface (CLI) tool for for running simulations locally.

Statement of need

To meet global climate targets, solar photovoltaic (PV) capacity must expand significantly. Tripling renewable energy capacity by 2030 is essential to limit global warming to 1.5°C (International Energy Agency, 2023). The expansion of PV plays a crucial role, and PV systems offer an additional benefit: small-scale house-mounted PV systems enable public participation and legitimize the energy transition. For calculating the yield of PV systems, various factors are important, including the location of the planned installation, local climate, surrounding objects such as houses or trees, and terrain. To provide accurate estimates of expected yields, simulation tools are essential in both research and practical PV system planning. For these reasons, a variety of software tools for simulating photovoltaic systems already exist (Holmgren, Hansen, Stein, et al., 2018; Jakica, 2018; Kumar Vashishtha et al., 2022).

`simshady` fills a gap in the area of detailed 3D shading simulation for PV yield estimation. This is especially important when modeling PV systems in real environments outside the laboratory, because shading caused by nearby objects, terrain, or other panels significantly lowers the yield of PV systems. Accurate shading simulation is therefore essential for real-world yield estimation.

State of the field

Existing software can roughly be grouped into three groups: full-suite commercial design tools, simplified yield calculators, and developer libraries.

Full-suite design tools target professional PV planners and combine 3D system modeling, component databases and financial analysis behind a graphical user interface. HelioScope by Aurora Solar is a web-based design platform that approximates near shading by projecting

drop shadows from 3D objects rather than performing full ray tracing, and is offered as a paid subscription (Aurora Solar, 2026). PV*SOL premium by Valentin Software is a Windows desktop application that performs detailed sub-module shading analysis at cell level, but is restricted to interactive single-project use, meaning it requires manual, GUI-based operation for one project at a time without automation or batch processing capabilities (Valentin Software GmbH, 2026). PVsyst is the industry reference for bankable yield reports and additionally provides a separately licensed command-line tool, PVsystCLI, which is the only existing software that combines accurate 3D near shading with batch automation, but at a cost of several thousand Swiss francs per year (PVsyst SA, 2026). The System Advisor Model (SAM) developed by NREL is open-source and exposes a Python wrapper (PySAM), but its 3D shading capabilities are limited compared to dedicated design tools (National Renewable Energy Laboratory, 2026b).

Among the developer libraries, the Python package pvlib (Anderson et al., 2023; Holmgren, Hansen, & Mikofski, 2018) offers a large range of functionalities. With pvlib it is possible to model shading scenarios where the whole 3D scene is not needed. Examples are the possibility to model horizon shading from far-away objects like hills, or modelling diffusive self-shading or partial module shading, to name a few. However, the topic of shading simulation with 3D objects is not included in this package. Another Python-based software that enables irradiance modeling in two dimensions is pvfactors (Anoma et al., 2017; *Pvfactors*, 2022), which computes view factors between PV rows and the ground to capture row-to-row and bifacial shading, but cannot represent arbitrary 3D obstacles such as trees or buildings. SoDeLe wraps pvlib in an end-user GUI for quick residential estimates and likewise omits 3D shading (Steinacker et al., 2026). Web-based tools for solar panel simulations, such as PVGIS, PVWatts, and RETScreen, provide an accessible means for non-technical individuals to estimate energy yields based on geographic location and building geometry (Psomopoulos et al., 2015). These tools account for far-shading from the horizon or terrain, or for self-shading derived from the ground coverage ratio, but lack the capability to perform shading simulations using 3D geometries from nearby objects.

Table 1 summarises the tools by the capabilities most relevant to urban-scale PV assessment. Two patterns emerge. First, every tool that supports detailed 3D shading is either commercial or restricted to interactive single-project use; the only freely scriptable option, PVsystCLI, is paywalled. Second, none of the open-source developer libraries can model arbitrary 3D shading, and none of the surveyed tools is designed for batch processing on large scales like simulating neighborhoods or whole cities.

Table 1: Comparison of representative PV simulation tools by the capabilities most relevant to automated, city-scale assessment. In the shading column, 3D means that 3-dimensional objects are used to model shading, 3D (simplified) means a 3D scene is used but objects are abstracted as polygons, 2D means that the simulation only relies on 2D abstractions, Horizon represents shading through the horizon contour, Row represents row-to-row self-shading without 3D objects, Diffuse represents reduction of diffuse irradiance due to the array geometry, and None means no shading modeling is provided.

Tool	Platform	Cost	Open Source	Shading	Interface	Source
HelioScope	Web	Paid	No	3D	API	(Aurora Solar, 2026)
PV*SOL prem.	Desktop	Paid	No	3D	No	(Valentin Software GmbH, 2026)
PVsyst	Desktop	Paid	No	3D	CLI	(PVsyst SA, 2026)
SAM	Desktop	Free	Yes	3D simplified	Py	(National Renewable Energy Laboratory, 2026b)
PVGIS	Web	Free	No	Horizon	API	(European Commission Joint Research Centre, 2026)

Tool	Platform	Cost	Open Source	Shading	Interface	Source
PVWatts	Web	Free	Yes	Row	API	(National Renewable Energy Laboratory, 2026a)
SoDeLe	Package	Free	Yes	None	Yes	(Steinacker et al., 2026)
pvlib	Package	Free	Yes	Horizon, Row, Diffuse	Yes	(Anderson et al., 2023; Holmgren, Hansen, & Mikofski, 2018)
pvfactors	Package	Free	Yes	2D	Yes	(Anoma et al., 2017; *Pvfactors*, 2022)

simshady now fills two existing gaps: First, it is implemented in TypeScript and hence simultaniously enables simulations in the browser and on a local machine. And second, it tackles the calculation intensive task of raytracing for shading simulation with a performant WebGL implementation on the GPU. The shader language GLSL (OpenGL Shading Language), on which WebGL is built, is not familiar for most developers: it only ranks 35th among programming languages on GitHub (GitHub, 2026). Hence, simshady makes the parallelized simulation of shading more accessible by exposing the WebGL simulation code through the more familiar JavaScript APIs instead.

Software design

simshady simulates the yield of photovoltaic (PV) systems by considering weather/climate data and shading from local 3D geometry. It is built around three core principles: (1) To run the core simulation code efficiently on the GPU, (2) to expose two different interfaces for both browser-based applications and server-based simulation, and (3) to integrate standardized data models, namely from `Three.js` and from the Wavefront OBJ file format.

Input data

In simshady, two major types of input data need to be provided. First, a 3D scene is built that represents the environment, consisting of primary objects for simulation (e.g., PV panels or target buildings) and surrounding objects that may cast shadows (e.g., neighboring buildings, trees). These objects can be provided as Wavefront .obj files or as Three.js geometries. Second, weather and climate data need to be provided as input data in .json format by aggregating irradiance data like Global Horizontal Irradiance (GHI) and Direct Normal Irradiance (DNI) datasets onto sky domes, for example by assigning irradiance values to each sky segment (Górski et al., 2005; Zonca et al., 2019).

Simulation pipeline

The central `ShadingScene` class orchestrates the simulation through the following steps:

1. **Geometry pre-processing:** First, the simulation mesh is refined by recursively subdividing triangles whose longest edge exceeds a configurable threshold (default 1.0 m), ensuring a uniform spatial resolution for the PV yield calculation. Reducing this threshold results in a larger number of smaller triangles in the mesh, and therefore a higher resolution of the simulated shading. To improve the simulation speed, a geometry filter removes shading triangles that cannot cast shadows on the simulation geometry given the minimum solar altitude angle present in the irradiance dataset.
2. **Ray tracing:** The simulation utilizes the Möller-Trumbore intersection algorithm (Möller & Trumbore, 1997) to determine if any shading objects obstruct the view between a sky

segment and the main simulation geometry (See Figure 1 a). For each triangle in the simulation geometry, a shading mask is generated, indicating whether an object blocks the incident irradiance beam from the sky dome surface to the triangle. The shading mask values range from 0 to 1, where 0 indicates that an object shades the triangle, 1 signifies that there is no obstruction and the line of sight is perpendicular to the triangle, and values between 0 and 1 represent cases where there is no obstruction but the angle of incidence is not perpendicular.

3. **Irradiance integration and yield calculation:** The irradiance values from all sky segments are needed as input data. They represent the irradiance in W/m^2 that reaches the simulation geometry from a given sky segment. They are multiplied by their corresponding shading mask values and summed to obtain the total irradiance received by each triangle. The resulting intensities are converted to electrical yield (in kWh/m^2) using a configurable solar-to-electricity conversion efficiency (default 15%). This efficiency includes the PV panel efficiency as well as the ratio of total area and area covered by PV panels.

4. **Visualization:** The computed yield values are normalized and mapped to RGB colors using a configurable colormap (See Figure 1 b). The resulting mesh carries both the color attribute for visualization and per-triangle solar yield for further analysis.

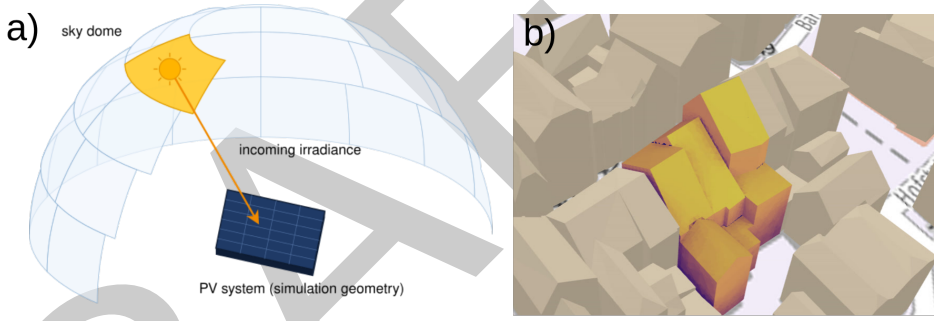

Figure 1: a) Schema of the simulation setup: For each surface of the sky dome and for each triangle of the simulation geometry, it is simulated if an object blocks the incoming irradiance. b) A simulated building with its yearly averaged solar yield, where dark purple represents low yields and light yellow represents high yields. Screenshot taken from (*Openpv.de*, 2024)

Simshady in the browser

`simshady` reuses the data model and objects of `Three.js`, a common package for 3D web applications. The package processes `Three.js` meshes and adds the simulated PV yield as a color to the mesh, as shown in Figure 1 b). Additionally, each triangle of the simulated buildings has its solar yield assigned as an attribute for further processing.

Simshady in the terminal

The `simshady` CLI is a wrapper around the core WebGL-based simulation engine that enables batch processing of photovoltaic-yield analyses on a server. It first parses a small set of required arguments (the simulation geometry and the irradiance data). The supplied geometry files, either JSON objects or Wavefront OBJ files, are handed to a headless Chromium instance launched via Puppeteer (Google Chrome Team, 2025).

Inside the browser context the full `simshady` package is injected, the scene is reconstructed, and the GPU-accelerated Möller-Trumbore ray-tracing routine is executed. When the calculation finishes, the CLI extracts the mesh data from the browser and writes the following artefacts into the user-specified output directory: binary files for vertex positions, colors, and intensities; a colour-coded Wavefront OBJ file with per-vertex RGB values; an orthographic top-down PNG

snapshot of the scene; and a JSON summary containing per-timestep and total aggregated yield statistics such as total and mean yield or surface area.

Research impact statement

Tool landscape positioning

`simshady` together with its CLI occupies a position in the PV-tool landscape that previously did not exist. Two simple matrices, derived from the comparison in Table 1, highlight this.

The first matrix groups tools by cost and 3D shading capability. Up to now, every tool that offered full 3D shading of arbitrary geometries was a commercial product, while every free or open-source tool either ignored 3D shading or was restricted to highly simplified geometric assumptions. Figure 2 shows this comparison.

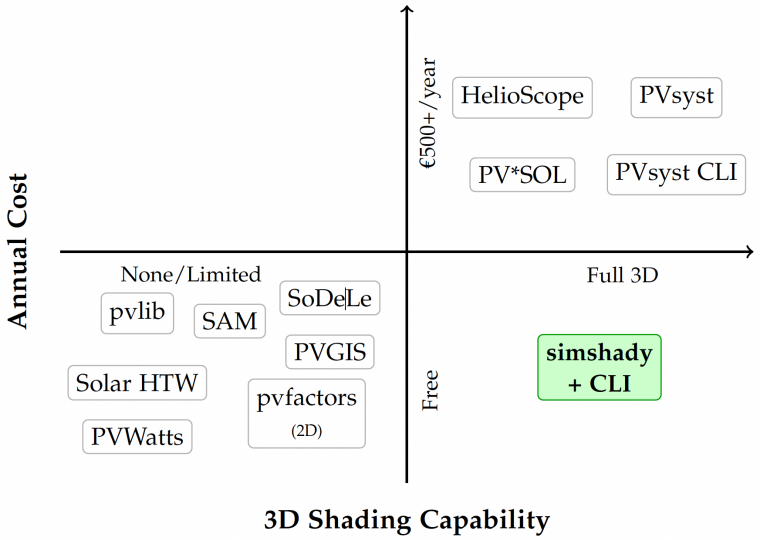

Figure 2: Tool positioning by cost and 3D shading capability. Free tools historically lacked 3D shading; 3D capability required commercial licenses. simshady with its CLI is the first free tool with a full 3D shading capability.

The second matrix groups the same tools by simulation scale and 3D shading capability. Tools that can be applied automatically to large numbers of buildings have so far skipped 3D shading entirely; tools with accurate 3D shading have been built around interactive single-project workflows. Figure 3 shows this comparison.

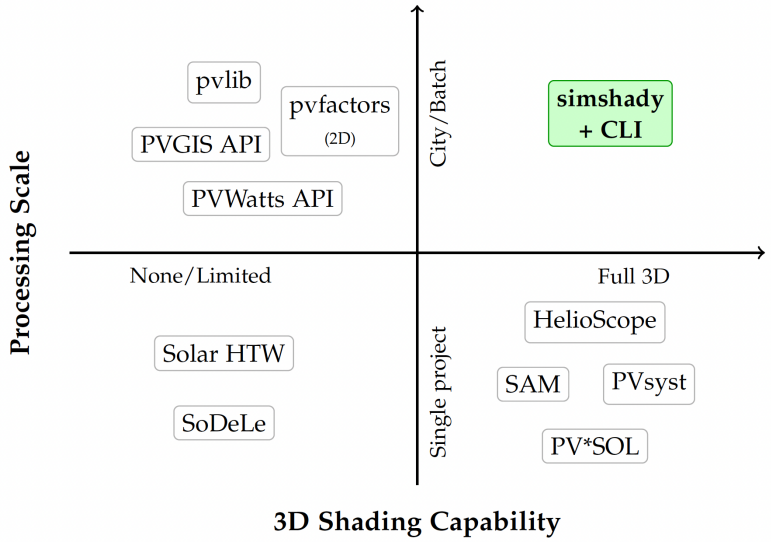

Figure 3: Tool positioning by simulation scale and 3D shading capability. City-scale PV assessment with accurate 3D shading was not possible before simshady and its CLI.

In both matrices the cell that `simshady` now occupies was empty before this work. `simshady` is therefore the first free and open-source tool with full 3D shading of arbitrary geometries, and the first such tool designed to handle both single buildings or entire cities.

City-scale demonstration: Munich

To show that this position is more than theoretical, the CLI was used to simulate the PV yield for the entire city of Munich. The original data consists of 109 CityGML LoD2 tiles. Each tile had a size of 2x2 km^2. For processing, each tiles was split into four tiles, one square kilometre each. The 3D data contained roughly 200 km^2 of building surface in total. Averaged incoming irradiance in W/m^2 per steradian was derived from the National Solar Radiation Database (Sengupta et al., 2018) and projected onto a HEALPix skydome. The skydomes together with the 3D geometries were then used for the ray-tracing simulation. Figure 4 a) shows one 3D mesh output for one tile. All tiles combined generate a simulation scene of the whole city of Munich, which is depicted in b) Figure 4 as an orthographic top-down image.

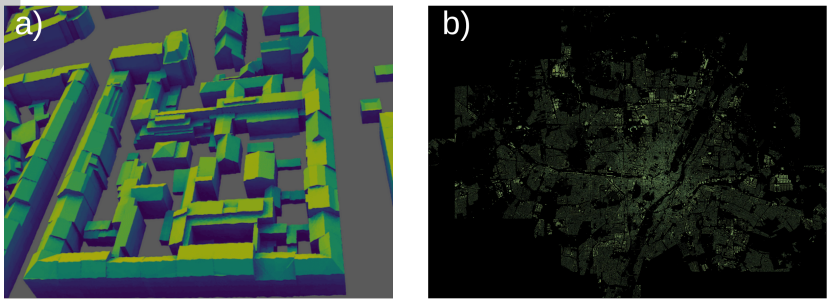

Figure 4: a) 3D mesh output of the CLI for a single Munich tile, with per-triangle annual PV yield mapped onto the building geometry (dark blue = low, light yellow = high). b) Orthographic top-down snapshots generated by the CLI for all Munich tiles, combined in one image.

The complete simulation took roughly 110 hours on a single cloud GPU server running four parallel CLI instances and cost less than 60 USD. The full output, including geometry, intensities,

metadata and visualisation artefacts for all tiles, makes up roughly 35 GB in compressed format and is openly available as a dataset on Zenodo under CC BY 4.0 (Estrella & Kotthoff, 2026).

Conclusion

The `simshady` package serves two primary purposes: it provides a solution for scientific calculations of PV yield on local machines through the CLI, while also facilitating science communication through interactive web-based simulations. The second one eliminates the need for specialized software or programming knowledge, making it accessible to a broader range of users.

Two design decisions are especially relevant: By implementing the main algorithm in WebGL, the package achieves higher performance than a pure JavaScript implementation, and it offers a JavaScript wrapper around PV simulations in WebGL. Additionally, `simshady` simplifies the simulation workflow by processing standardized input formats, thereby simplifying its integration in existing workflows.

In the future, `simshady` can profit from various developments: First, the architectural decision of outsourcing the calculation of irradiance has a clear benefit. If averaged irradiance data is provided for a whole year, simshady calculates the averaged PV yield per year. If time series of irradiance data are provided, simshady can also simulate time series of PV yield that show the local shading influence at highly resolved times. However, with this benefit comes the disadvantage that users need to provide irradiance data themselves. One useful extension would be to integrate the creation of these time series into the software.

Second, the interoperability with existing standards is started in `simshady`, but not finished. Processing three.js objects or .obj files is already nice, but providing results in formats that can be reused in other software tools, like `pvlib` for example, would also be beneficial for the whole community.

To extend and maintain `simshady` in the future, other researchers and developers are warmly welcome to contribute to the codebase or discuss new ideas within the issue section.

Credit Authorship Statement

FK: Conceptualization, Software, Funding acquisition, Writing – original draft

MG: Conceptualization, Software, Funding acquisition, Writing – review & editing

KH: Conceptualization, Software, Funding acquisition, Writing – review & editing

ME: Software, Writing – review & editing

KP: Conceptualization, Software, Funding acquisition, Writing – review & editing

AI usage disclosure

After designing the software architecture and writing core functionalities, both open-weight and proprietary LLM-based chatbots where used to implement or debug some of the methods in `simshady`. Generative agentic AI tools that can write and edit code autonomously were not used. The open-weight models gpt-oss:120b and deepseek-r1:70b were used for reviewing the written text of this paper. Individual parts of the svg file for Figure 1 where created with the help of a proprietary LLM.

Acknowledgements

The development of this software was funded by the German Federal Ministry of Research, Technology and Space within the "Software Sprint - Support for Open Source Developers" program by Prototype Fund.

References

Anderson, K. S., Hansen, C. W., Holmgren, W. F., Jensen, A. R., Mikofski, M. A., & Driesse, A. (2023). Pvlib python: 2023 project update. *Journal of Open Source Software*, *8*(92), 5994. https://doi.org/10.21105/joss.05994

Anoma, M. A., Jacob, D., Bourne, B. C., Scholl, J. A., Riley, D. M., & Hansen, C. W. (2017). View factor model and validation for bifacial PV and diffuse shade on single-axis trackers. *2017 IEEE 44th Photovoltaic Specialist Conference (PVSC)*, 1549–1554. https://doi.org/10.1109/PVSC.2017.8366704

Aurora Solar. (2026). *HelioScope solar design software*. https://helioscope.aurorasolar.com/.

Estrella, M., & Kotthoff, F. (2026). *simshady munich PV yield simulation dataset (CityGML LoD2, 432 tiles)*. Zenodo. https://doi.org/10.5281/zenodo.18742162

European Commission Joint Research Centre. (2026). *PVGIS – photovoltaic geographical information system*. https://re.jrc.ec.europa.eu/pvg_tools/en/.

GitHub. (2026). *GitHub Innovation Graph*. https://github.com/github/innovationgraph

Google Chrome Team. (2025). *Puppeteer: Headless chrome node.js API* (Version v24.31.0). Google. https://github.com/puppeteer/puppeteer

Górski, K. M., Hivon, E., Banday, A. J., Wandelt, B. D., Hansen, F. K., Reinecke, M., & Bartelmann, M. (2005). HEALPix: A framework for high-resolution discretization and fast analysis of data distributed on the sphere. *The Astrophysical Journal*, *622*(2), 759. https://doi.org/10.1086/427976

Holmgren, W. F., Hansen, C. W., & Mikofski, M. A. (2018). Pvlib python: A python package for modeling solar energy systems. *Journal of Open Source Software*, *3*(29), 884.

Holmgren, W. F., Hansen, C. W., Stein, J. S., & Mikofski, M. A. (2018). Review of open source tools for PV modeling. *2018 IEEE 7th World Conference on Photovoltaic Energy Conversion (WCPEC)(a Joint Conference of 45th IEEE PVSC, 28th PVSEC & 34th EU PVSEC)*, 2557–2560. https://doi.org/10.1109/PVSC.2018.8548231

International Energy Agency. (2023). *Tripling renewable power capacity by 2030 is vital to keep the 1.5°c goal within reach*. https://www.iea.org/commentaries/tripling-renewable-power-capacity-by-2030-is-vital-to-keep-the-150c-goal-within-reach.

Jakica, N. (2018). State-of-the-art review of solar design tools and methods for assessing daylighting and solar potential for building-integrated photovoltaics. *Renewable and Sustainable Energy Reviews*, *81*, 1296–1328. https://doi.org/10.1016/j.rser.2017.05.080

Kumar Vashishtha, V., Yadav, A., Kumar, A., & Kumar Shukla, V. (2022). An overview of software tools for the photovoltaic industry. *Materials Today: Proceedings*, *64*, 1450–1454. https://doi.org/10.1016/j.matpr.2022.04.737

Möller, T., & Trumbore, B. (1997). Fast, minimum storage ray-triangle intersection. *Journal of Graphics Tools*, *2*(1), 21–28. https://doi.org/10.1080/10867651.1997.10487468

National Renewable Energy Laboratory. (2026a). *PVWatts calculator*. https://pvwatts.nrel.gov.

National Renewable Energy Laboratory. (2026b). *System advisor model (SAM)*. https://sam.nrel.gov.

Openpv.de. (2024). www.openpv.de.

Psomopoulos, C. S., Ioannidis, G. C., Kaminaris, S. D., Mardikis, K. D., & Katsikas, N. G. (2015). A comparative evaluation of photovoltaic electricity production assessment software (PVGIS, PVWatts and RETScreen). *Environmental Processes*, *2*, 175–189. https://doi.org/10.1007/s40710-015-0092-4

Pvfactors: Open-source view-factor model for diffuse shading and bifacial PV modeling (Version 1.5.2). (2022). https://github.com/SunPower/pvfactors

PVsyst SA. (2026). *PVsyst photovoltaic software*. https://www.pvsyst.com.

Sengupta, M., Xie, Y., Lopez, A., Habte, A., Maclaurin, G., & Shelby, J. (2018). The national solar radiation data base (NSRDB). *Renewable and Sustainable Energy Reviews*, *89*, 51–60.

Steinacker, H., Mucke, J., Ott, E., & Stickel, M. (2026). *SoDeLe: Solarsimulation denkbar leicht*. https://github.com/QuaSi-Software/SoDeLe.

Valentin Software GmbH. (2026). *PV*SOL premium – planning and simulation of photovoltaic systems*. https://valentin-software.com/en/products/pvsol-premium/.

Zonca, A., Singer, L., Lenz, D., Reinecke, M., Rosset, C., Hivon, E., & Gorski, K. (2019). Healpy: Equal area pixelization and spherical harmonics transforms for data on the sphere in python. *Journal of Open Source Software*, *4*(35), 1298. https://doi.org/10.21105/joss.01298